

National University of Computer and Emerging Sciences, Lahore Campus

	Course Name:	Information Security	Course Code:	CS3002
	Program:	BCS	Semester:	Fall 2026
	Section:	7C	Total Marks:	100
	Due Date:	13-Sep-2026	Weight	
	Exam Type:	Assignment 1	Page(s):	12

Operation Lost Drive

Cryptography Investigation

Assignment

Contents

National University of Computer and Emerging Sciences, Lahore Campus.....	1
1 Assignment Overview	2
2 Personalisation and Fairness	2
3 What the Student Requires.....	3
3.1 Software and equipment.....	3
3.2 Files received.....	3
4 How to Approach the Investigation.....	3
5 Required Technical Work.....	4
5.1 Monoalphabetic substitution program.....	4
5.2 Vigenere program.....	5
5.3 Rail Fence program	5
5.4 A–P byte-decoding and final XOR program.....	5
6 Evidence Log and Investigation Report.....	6
7 Required Submission	6
7.1 Identity file.....	7
7.2 Flags file.....	7
7.3 README file.....	7
8 Theoretical Questions	7
9 Allowed and Prohibited Activities.....	8
9.1 Allowed	8
9.2 Prohibited	8
10 Marking Rubric	9
11 Submission and Verification Process.....	9
11.1 Student submission process	9
Program Execution Requirement.....	9

1 Assignment Overview

Each student receives a personalised ZIP archive representing the readable contents of a recovered USB drive. The fictional organisation was preparing for a system migration when an employee created a recovery archive. Relevant information is distributed across current documents, historical records, message exports, backup material, operational logs, and an archive folder.

This is an evidence-led investigation. It is not a collection of four separately labelled ciphertext questions. Before decrypting anything, students must establish which record is relevant, which version applies, and how one recovered result relates to another record. Every important conclusion must be supported by evidence from the supplied files.

The investigation uses the following practical cryptographic topics:

1. monoalphabetic substitution analysis;
2. Vigenere decryption;
3. standard zigzag Rail Fence decryption and depth testing; and
4. byte-wise XOR using a correctly reconstructed one-time-pad key.

AES, DES, confusion, diffusion, and one-time-pad key reuse are assessed only through short conceptual questions. Students are not required to implement AES or DES or to learn unrelated forensic software.

Central principle: Do not select a file merely because its name looks important. Use record identifiers, internal timestamps, version information, and the content of recovered messages to justify each next action. Filesystem modification times and ZIP timestamps are not evidence.

2 Personalisation and Fairness

Every student receives a different assignment instance generated from their complete normalised roll number, an instructor-only master secret, and independent random recovery material. Depending on the instance, the following differ between students:

- employee, project, message, transfer, dispatch, and batch references;
- internal dates and the locations of supporting records;
- substitution passage and shuffled substitution alphabet;
- historical and current project keywords;
- Vigenere and Rail Fence ciphertexts;
- the actual Rail Fence depth;
- all recovery components and stage tokens; and
- the final ciphertext, case reference, confidential message, and final token.

Another student's ciphertexts, keys, evidence references, and tokens will not answer your assignment. Generic knowledge of the algorithms may still be discussed subject to the course integrity policy. Personalisation does not make the archive "AI-proof", "cheat-proof", or secure storage.

3 What the Student Requires

3.1 Software and equipment

Students require only:

- Python 3.11 or newer;
- a text editor or IDE;
- an application capable of extracting an ordinary ZIP archive;
- a PDF reader; and
- normal tools for reading TXT, CSV, JSON, LOG, Markdown, and HEX files.

No website, web server, browser investigation, external API, paid service, specialist forensic suite, disk image, or Internet connection is required.

3.2 Files received

Each student receives exactly one ZIP archive named with their roll number and assignment version, for example:

```
Operation_Lost_Drive_22L-7776_v1.zip
```

After extraction, the top-level contents include:

```
START_HERE.txt
Assignment_Brief.pdf
Assignment_Brief.md
identity.json
evidence_log.csv
submission_examples.txt
Documents/
Messages/
Backups/
Logs/
Archive/
```

The roll number and assignment version in identity.json must match the student and the issued ZIP. A student who receives the wrong archive must not begin solving it and must contact the instructor.

4 How to Approach the Investigation

Students should use a repeatable investigation process rather than opening files at random.

1. Read START_HERE.txt and the complete assignment brief before beginning.
2. Extract the complete archive into a normal folder. Do not work directly inside the ZIP.
3. Preserve an unchanged copy of the extracted archive.
4. Make a brief inventory of the supplied directories and record types.
5. Read records carefully and compare their internal timestamps, statuses, versions, and identifiers.

6. Treat each conclusion as an evidence claim: record the source, the observed fact, the inference, and the next action.
7. When encrypted material becomes relevant, consult the supplied communication-conventions record and implement the stated operation in Python.
8. After decryption, read the complete plaintext. A recovered result may supply a token, a recovery component, and information needed to interpret another record.
9. Verify every structured value before using it. Check spelling, letter case, encoding rules, byte lengths, and component order.
10. Record plausible alternatives and explain why the evidence rejects them.
11. Retain commands and intermediate output so the work can be reproduced in the report.

Students are expected to ask questions such as:

- Which record does this identifier refer to?
- Is this record current, historical, failed, superseded, or applicable at the relevant time?
- What cryptographic convention does the supplied evidence establish?
- Does the recovered plaintext make linguistic and structural sense?
- What new evidence can be evaluated using the recovered result?

These questions explain the required method without identifying the correct files, references, keys, mappings, depths, components, or route for any student.

5 Required Technical Work

5.1 Monoalphabetic substitution program

The submitted `mono_solver.py` must:

- accept a ciphertext file as an external command-line input;
- calculate and print an A–Z letter-frequency table;
- accept a student-derived partial or complete cipher-to-plaintext mapping;
- reject an invalid mapping that assigns characters inconsistently;
- preserve spaces, punctuation, digits, and line structure; and
- print the plaintext produced by applying the supplied mapping.

The program is not required to be a fully automatic substitution cracker. The report must show how frequencies, repeated patterns, the located known-word information supplied in the archive, and consistency across the passage contributed to the mapping. A long passage alone is not proof of a correct solution.

One acceptable command-line design is:

```
python mono_solver.py --input selected_message.txt --mapping mapping.json
```

The mapping format must be documented in the submitted `README.txt`.

5.2 Vigenere program

The submitted `vigenere_solver.py` must:

- accept an external ciphertext file and keyword;
- reject an empty keyword or a keyword containing nonletters;
- use uppercase ASCII letters with A=0 through Z=25;
- preserve spaces and punctuation;
- advance the key only for A–Z characters;
- implement plaintext recovery using $P = (C - K) \bmod 26$; and
- print the complete calculated plaintext.

An acceptable interface is:

```
python vigenere_solver.py --input selected_dispatch.txt --keyword WORD
```

The report must explain how the keyword and applicable record version were selected from evidence. Merely trying words until one appears readable is not a complete justification.

5.3 Rail Fence program

The submitted `rail_solver.py` must:

- accept a ciphertext file as an external command-line input;
- implement the standard top-to-bottom, down-and-up zigzag Rail Fence algorithm;
- preserve the entire character sequence under the documented line-ending convention;
- print clearly labelled candidate plaintexts for every depth from 2 through 8; and
- allow the meaningful result to be selected from evidence rather than hard-coding one depth or plaintext.

An acceptable interface is:

```
python rail_solver.py --input selected_payload.txt
```

The report must include all seven candidate depths and justify the selected result using readable content and valid structured recovery data.

5.4 A–P byte-decoding and final XOR program

Sensitive byte values recovered through the classical stages use the public A–P nibble encoding described in the supplied conventions. A through P represent nibble values 0 through 15. Two letters represent one byte, with the high nibble first. This is an encoding, not encryption.

The submitted `otp_solver.py` must:

- accept the final ciphertext and the three recovered A–P components as external inputs;
- reject odd-length A–P data and characters outside A–P;
- decode each pair of A–P characters into one byte;

- concatenate the decoded components in the order stated by the supplied recovery convention;
- validate hexadecimal ciphertext and reject an odd number of hexadecimal digits;
- confirm that the reconstructed key and ciphertext have equal byte lengths;
- XOR corresponding bytes exactly once; and
- decode and print the recovered UTF-8 message.

An acceptable interface is:

```
python otp_solver.py --ciphertext final_cipher.hex \  
--alpha AP_COMPONENT --beta AP_COMPONENT --gamma AP_COMPONENT
```

Do not repeat a short component to fill the key. Do not describe the A–P representation as encryption. The final operation is byte-wise XOR, not AES or DES.

6 Evidence Log and Investigation Report

The supplied evidence_log.csv template uses these fields:

```
stage,source path,record/line reference,observed clue,inference,next action,result
```

Use paths relative to the extracted assignment root so they remain meaningful after submission. Refer to record identifiers, CSV rows, JSON objects, or line labels rather than relying only on screenshots.

The PDF report should be approximately three to five concise pages and use the following headings:

1. Student name, roll number, section, assignment version, and assignment title.
2. Investigation approach and evidence-selection process.
3. Monoalphabetic frequency table, mapping development, known-word use, and verification.
4. Historical/version reasoning and Vigenere decryption.
5. Rail Fence candidates for depths 2 through 8 and selection justification.
6. A–P component decoding, byte lengths, key reconstruction, XOR procedure, and final result.
7. At least two plausible alternatives rejected using evidence.
8. Answers to the conceptual questions in Section 8.
9. Brief conclusion and assistance disclosure.

Screenshots may support an explanation, but screenshots alone are not a substitute for reasoning, evidence references, source code, or reproducible commands.

7 Required Submission

Submit one ZIP named using the following convention:

```
ROLL-NUMBER_Operation-Lost-Drive_Submission.zip
```

The ZIP must contain one submission/ folder with exactly these required files:

```
submission/  
|-- identity.json  
|-- mono_solver.py  
|-- vigenere_solver.py  
|-- rail_solver.py  
|-- otp_solver.py  
|-- flags.txt  
|-- evidence_log.csv  
|-- report.pdf  
'-- README.txt
```

Do not submit a virtual environment, cache directory, installed package, unrelated screenshot collection, instructor material, or another copy of the original assignment ZIP.

7.1 Identity file

The UTF-8 identity.json must contain exactly the student's normalised roll number and issued assignment version:

```
{  
  "roll_number": "22L-7776",  
  "assignment_version": 1  
}
```

Replace the example values with the values from the issued assignment. Do not rename the fields or add comments; JSON does not support comments.

7.2 Flags file

The UTF-8 flags.txt file must use this exact structure:

```
ALPHA=YOUR-RECOVERED-ALPHA-TOKEN  
BETA=YOUR-RECOVERED-BETA-TOKEN  
GAMMA=YOUR-RECOVERED-GAMMA-TOKEN  
FINAL=YOUR-RECOVERED-FINAL-TOKEN
```

Replace every descriptive value with the token recovered from the student's own assignment. Tokens are case-sensitive. Do not add quotation marks, spaces around the equals sign, renamed fields, duplicated labels, or commentary on token lines.

7.3 README file

The submitted README.txt must state:

- the exact command used to run each solver;
- the required Python version;
- the mapping-file format used by the monoalphabetic solver; and
- a truthful disclosure of human, online, software, or AI assistance used during the assignment.

8 Theoretical Questions

Answer each question concisely in the report. AES and DES are theory-only topics in this assignment.

1. Why do English-language frequency patterns help when analysing a monoalphabetic substitution, and why is frequency alone not proof of a mapping?
2. Why does Rail Fence transposition preserve the frequency count of every character?
3. If the same one-time-pad key encrypts two messages, what relationship can an attacker calculate from the two ciphertexts? What does this relationship not automatically reveal?
4. Why is single DES considered obsolete for modern protection?
5. What do confusion and diffusion mean in modern cipher design, and why should a simple transposition not be described as strong modern diffusion?

No AES or DES implementation is required or accepted as a substitute for these explanations.

9 Allowed and Prohibited Activities

9.1 Allowed

- Python 3.11 or newer and its standard library.
- A text editor or IDE.
- Ordinary ZIP, PDF, text, CSV, JSON, and hexadecimal viewing tools.
- Course slides, textbooks, and general references about the relevant algorithms.
- Legitimate alternative solving methods when they are explained and supported by evidence.
- Assistance permitted by the instructor's reviewed course policy and disclosed in README.txt.

9.2 Prohibited

- Sharing or copying personalised ciphertexts, plaintexts, record references, mappings, keywords, components, tokens, reports, or source code.
- Submitting another student's ZIP, answers, programs, evidence log, or report content.
- Presenting hard-coded plaintext or tokens as a cryptographic implementation.
- Altering identity or assignment-version information to use answers from another instance.
- Attacking institutional systems, accounts, networks, email, cloud permissions, or LMS infrastructure.
- Implementing AES or DES instead of completing the required investigation and classical-cipher work.

Academic-integrity and AI policy — instructor review required before release:

The default policy permits tools or AI assistance only when it is disclosed in README.txt and the student can explain every submitted inference and program. AI-detection claims will not be used as evidence of misconduct. The instructor may replace this paragraph with the institution's approved policy before issuing the assignment.

10 Marking Rubric

Assessment Area	Marks
Evidence discovery, correlation, and rejected alternatives	25
Monoalphabetic analysis and working solver	20
Vigenere key selection and working solver	15
Rail Fence search and justified recovery	10
OTP reconstruction and working decryption	15
Four correct tokens: ALPHA 2, BETA 2, GAMMA 2, FINAL 4	10
Short conceptual reflections	5
Total	100

The automatic grader checks only the four token fields, worth 10 marks. It also reports missing files and invalid identity information. The remaining 90 marks require manual evaluation of the evidence, reasoning, programs, report, alternatives, and conceptual answers. An incorrect early result does not automatically remove all marks from later sound work.

11 Submission and Verification Process

11.1 Student submission process

1. Run each program from a clean copy of the submission folder using the commands documented in README.txt.
2. Confirm that each solver accepts external input and performs the required calculation rather than printing only a fixed answer.
3. Confirm that identity.json contains the correct roll number and assignment version.
4. Confirm that flags.txt contains all four nonempty tokens in the required format.
5. Open evidence_log.csv and confirm that paths, record references, observations, inferences, and results are readable.
6. Open report.pdf and confirm that every page is readable.
7. Confirm that the final submission ZIP contains exactly one submission/ folder and the nine required files.
8. Upload the submission before the stated due date.

Program Execution Requirement

Each submitted Python program must be executable from the submission folder using the command documented in README.txt. Programs must use command-line arguments for ciphertext files, mappings, keywords, or components as applicable. The instructor should not need to edit source code before running a documented command.

Each program should use the following structure:

```
def main():  
    # Parse and validate command-line inputs
```

```
# Perform the required cryptographic calculation
# Print clearly labelled calculated output
if __name__ == "__main__":
    main()
```

When executed with valid documented inputs, each program must:

- finish without an unhandled error;
- return exit status zero;
- print clearly labelled output;
- perform the required calculation;
- print the calculated recovered plaintext; and
- produce the same result on repeated execution with the same inputs.

Programs that only print hard-coded plaintext or token values without performing the required calculation will not receive implementation marks, even if the printed answers are correct. Student programs are not executed by the ordinary automatic token grader; code is reviewed separately under the instructor's approved safe procedure.

Important Submission Instructions

Follow the filenames, folder structure, identity format, and token format exactly. Automated checks may report a failure when required files are renamed, misplaced, corrupted, or saved in the wrong format, even when part of the investigation is correct.

Your submission ZIP must contain these nine files inside one submission/ folder:

1. identity.json
2. mono_solver.py
3. vigenere_solver.py
4. rail_solver.py
5. otp_solver.py
6. flags.txt
7. evidence_log.csv
8. report.pdf
9. README.txt

The flags.txt file must be plain UTF-8 text with these fields in this exact order:

```
ALPHA=YOUR-RECOVERED-ALPHA-TOKEN  
BETA=YOUR-RECOVERED-BETA-TOKEN  
GAMMA=YOUR-RECOVERED-GAMMA-TOKEN  
FINAL=YOUR-RECOVERED-FINAL-TOKEN
```

Before submitting, confirm that:

- every documented program command executes without an error;
- all required inputs are passed externally and can be found;
- mappings, keywords, encodings, hexadecimal values, and byte lengths are validated;
- identity.json matches the issued assignment;
- flags.txt contains the student's own four recovered tokens;
- evidence_log.csv and report.pdf open correctly; and
- the final ZIP contains no virtual environment, cache directory, original evidence archive, or unrelated file.

If you have questions about the submission format, contact the instructor before the deadline at:

l227776@lhr.nu.edu.pk

:-)